INFO0952 - Compléments d'informatique

Exemples de questions sur les TP 1 et 2

6 octobre 2026

Connaissance générale du C

1. Quelle instruction ci-dessous correspond à l'allocation d'un tableau de 10 chaînes de caractères ?

   ☐ `malloc(10 * sizeof(char*))`

   ☐ `malloc(10 * sizeof(char))`

   ☐ `malloc(sizeof(10 * char))`

   ☐ `malloc(10)`

2. Soit le code suivant :

```
1  void myf1(void *data)
2  {
3      (((int *)data)[0])++;
4  }
5
6  int myf2(void (*f)(void *))
7  {
8      int v[1] = {3};
9      f(v);
10     return v[0];
11 }
```

   Que va renvoyer l'appel `myf2(myf1)` ?

   ☐ `2`

   ☐ `3`

   ☐ `4`

   ☐ Une valeur indéterminée.

   ☐ Le code ne compilera pas/créera une segmentation fault.

3. Soit deux fichiers `bidule.c` et `bidule.h`. La ligne suivante se trouve dans `bidule.h` :

```
1 typedef struct Point_t Point;
```

et le code suivant se trouve dans `bidule.c` :

```
1 #include "bidule.h"
2
3 struct Point_t {
4     int x;
5     int y;
6 }
```

Indiquez pour chacun des bouts de code ci-dessous s'ils sont permis (c'est-à-dire si aucune erreur ne sera produite à la compilation) dans le fichier `bidule.c` et/ou dans un fichier `main.c` qui inclurait `bidule.h` ?

```
1 Point *p = malloc(sizeof(Point));
2 p->x = 4;
```

☐ Permis dans `bidule.c`
☐ Permis dans `main.c`
☐ Les deux
☐ Aucun des deux

```
1 Point p;
2 p.x = 4;
```

☐ Permis dans `bidule.c`
☐ Permis dans `main.c`
☐ Les deux
☐ Aucun des deux

```
1 Point *p1 = malloc(sizeof(Point));
2 Point p2;
3 p2.x += p1.x;
```

☐ Permis dans `bidule.c`
☐ Permis dans `main.c`
☐ Les deux
☐ Aucun des deux

TP 1

1. Avez-vous utilisé la fonction `malloc` dans votre solution ?
   ☐ Oui
   ☐ Non
2. Soit deux tableaux x et y de type `int` et de longueurs 10. Il y a une paire discordante entre les tableaux s'ils existent deux entiers `i` et `j` tels que :
   ☐ `x[i] < y[i]` et `x[j] > y[j]`
   ☐ `x[i] < x[j]` et `y[i] > y[j]`
   ☐ `x[i] < y[j]` et `x[j] > y[i]`

3. Quelle est la valeur du taux de Kendall entre ces deux tableaux ?

```
x = [0, 1, 4]
y = [0, 2, 1]
```

   - ☐ 1/2
   - ☐ 1/3
   - ☐ -2/3

4. Est-ce que la fonction `kendallTauSlow` qu'on vous a fournie fonctionne dans le cas où il y a des doublons dans les tableaux ?
   - ☐ Oui.
   - ☐ Uniquement s'il n'y a pas de doublons dans le tableau `x`.
   - ☐ Uniquement s'il n'y a pas de doublons dans le tableau `y`.
   - ☐ Non.

5. Est-ce que la fonction `kendallTauFast` qu'on vous a demandé d'implémenter fonctionne dans le cas où il y a des doublons dans les tableaux ?
   - ☐ Oui.
   - ☐ Uniquement s'il n'y a pas de doublons dans le tableau `x`.
   - ☐ Uniquement s'il n'y a pas de doublons dans le tableau `y`.
   - ☐ Non.

6. Est-ce que les appels suivants renverront les mêmes valeurs ?

```
kendallTauFast(x,y)
kendallTauFast(y,x)
```

   - ☐ Oui, toujours.
   - ☐ Non, pas nécessairement.

7. Quelle est la complexité en espace de `kendallTauSlow` en fonction de la longueur des tableaux notée N ?
   - ☐ $O(1)$
   - ☐ $O(N)$
   - ☐ $O(N \log(N))$
   - ☐ $O(N^2)$

8. Quelle est la complexité en espace de `kendallTauFast` en fonction de la longueur des tableaux notée N ?
   - ☐ $O(1)$
   - ☐ $O(N)$
   - ☐ $O(N \log(N))$
   - ☐ $O(N^2)$

TP2

9. Pourquoi le tableau tab est-il de type `void *` dans la fonction `findFirstTrue` ?
   - ☐ Pour rendre le type du tableau opaque.
   - ☐ Pour empêcher la modification des éléments stockés dans le tableau.
   - ☐ Pour accepter n'importe quel type de tableau.
   - ☐ Pour indiquer que le tableau contient exclusivement des pointeurs.
10. À quoi sert le paramètre `void *param` passé à la fonction `findFirstTrue` ?
    - ☐ À indiquer le nombre total d'éléments dans le tableau `tab`.
    - ☐ À stocker la valeur de retour calculée par la fonction `f`.
    - ☐ À transmettre des données contextuelles à la fonction `f`.
    - ☐ À vérifier que le pointeur de fonction `f` n'est pas égale à `NULL`.
11. Dans le cas où `tab` est un tableau de `float`, est-ce que la fonction `f` suivante est correcte pour trouver la première valeur strictement positive dans le tableau supposé trié ? Si non, corrigez l'erreur.

```
1 bool f(void *tab, int i, void *param)
2 {
3     return (float)tab[i] > 0.0;
4 }
```

- ☐ Oui
- ☐ Non

12. Que va afficher le code suivant :

```
1 bool fo(void *tab, int i, void *param)
2 {
3     int length = *(int *)param;
4     char *tabp = (char *)tab;
5     return tab[length-1-i] == 'c';
6 }
7 char *tab = "cccooooo";
8 int length = 8;
9 printf("%d\n", findFirstTrue(fo, tab, length, &length));
```

- ☐ Rien. Le code ne compilera pas.
- ☐ 0
- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5

13. Ecrivez une version naïve de `findFirstTrue` de complexité $O(N)$: